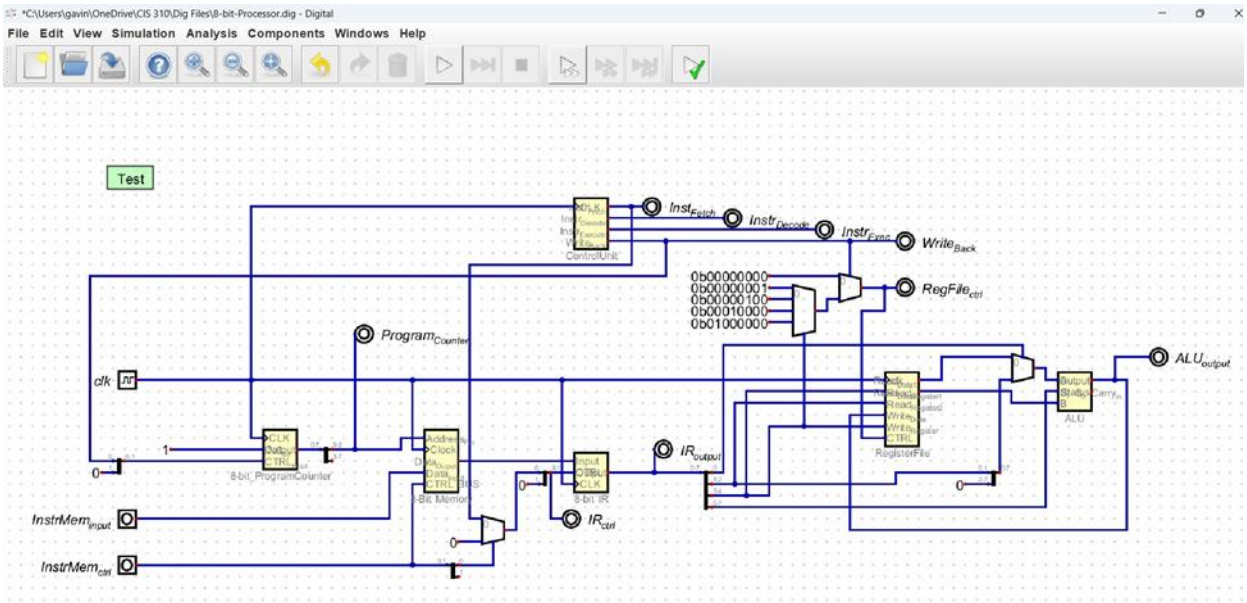
CIS 310 — Assignment 3

8-Bit Processor

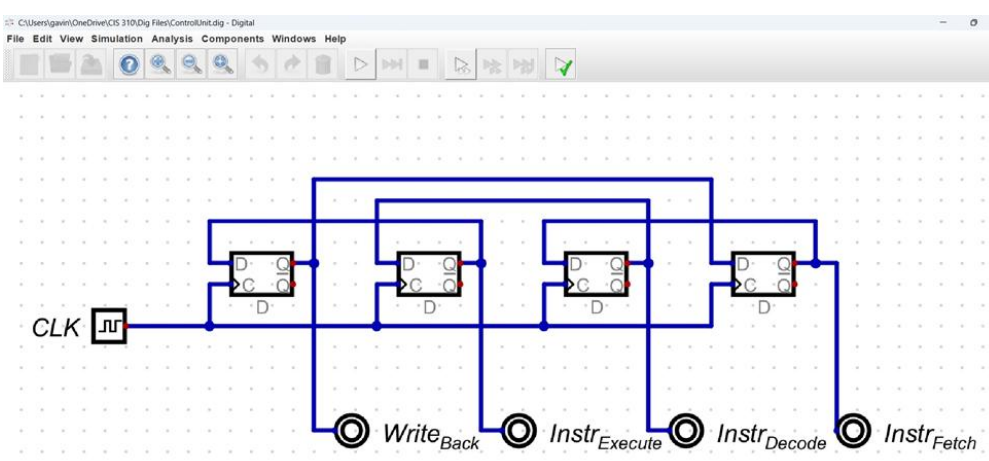
Gavin Llewellyn

Block Diagram



The top-level datapath shows the 3 stage layout that is described in the Assignment 3 requirements. Stage 1 (starting at the left side) is Instruction Fetch: the Program Counter drives the 8-word instruction memory, the output then feeds the 8-bit IR. Stage 2 (middle) is Instruction Decode: a splitter breaks the IR output into four fields — opcode [7:5], op1 [4:3], op2 [2:1], and type [0]. Stage 3 (right) is Execution/Writeback: the Register File and ALU compute the result, and a gated control bus selects which register receives the ALU output during WRITEBACK. The Control Unit sits at the top center with 4 states. The 3 opcode bits are wired directly to the ALU's control input without decoding logic. Also, there's an immediate path as for I-type instructions, op2 is 8 bits and routed through a 2-to-1 MUX into ALU input A in place of Read_Data1, so only one extra MUX compared to an R-type-only machine.

Four-State FSM Explanation



The Control Unit is one counter made from four D Flip-Flops arranged right-to-left. Instr_Fetch is initialized to 1, the other three default to 0. Each clock pulse shifts the single logic 1 by 1 position to the left. Therefore, the state sequence is Fetch – Decode – Execute – Writeback – Fetch - Decode… with only 1 output active at all times. The flip-flop on the left's Q output follows the top wire back to the flip-flop on the right's D input, closing the circuit.

The four state signals drive control multiplexers throughout the datapath. Also, the IR-control MUX is driven by Instr_Fetch only when the processor is in run mode. When InstrMem_ctrl = 01, the MUX forces IR_ctrl = 00 so the IR doesn't grab the instructions being written into memory. Additionally, the RegisterFile control bus is gated by Write_Back. Besides the WRITEBACK state, all 8 control bits are forced to 0 so every register holds its value, and only during WRITEBACK does the 4-to-1 MUX send 01 into the two control bits of the destination register.

Execution Flow

Consider the execution of SUB R1, R2 (encoding 0x6C, binary 01101100). For Fetch, the IR_ctrl transitions to 01 for one clock pulse. The IR captures 0x6C from instruction memory at the current PC address. For Decode, The IR-output splitter processes opcode = 011, op1 = 01 (R1), op2 = 10 (R2), type = 0. The opcode is routed to the ALU control input. op1 starts Read_Register1 and the 4-to-1 MUX selector; op2 drives Read_Register2. The RegisterFile produces Read_Data1 = R1 and Read_Data2 = R2. For Execute, since type = 0, the ALU A-input MUX selects Read_Data1. The ALU computes Output = A + Y + Cin (and S1, S0, Cin = 011), which evaluates Y = ~B and adds Cin = 1, resulting in A + (~B) + 1 = A − B. For Writeback, Write_Back goes high. The 4-to-1 MUX (from op1 = 01), outputs 0b00000100, which sets the two control bits of R1 to 01 (store). The RegisterFile writes the ALU output into R1. At the same time PC.CTRL[0] = Write_Back advances the PC to the next instruction.

Test Results

The test script test_8bit_processor.txt was loaded into the Digital circuit test code. The script programmed seven instructions into memory: INC R1 at 0-3, INC R2 at 4-5, and SUB R1 and R2 at address 6, then running 40 clock cycles and comparing each signal value to expected reference data. The test reported as "passed" and every row is highlighted in green. **(See Screenshots Below)**

Test result

File View

passed

	clk	Program_...	InstrMem...	InstrMem...	IR_output	IR_ctrl	Write_Back	ALU_output
L78	0	0	A8	1	0	0	0	0
L79;n=0	0	0	0	0	0	0	0	0
L79;n=1	0	0	0	0	0	0	1	0
L79;n=2	0	1	0	0	0	1	0	0
L88	0	1	A8	1	0	0	0	0
L89;n=0	0	1	0	0	0	0	0	0
L89;n=1	0	1	0	0	0	0	1	0
L89;n=2	0	2	0	0	0	1	0	0
L91	0	2	A8	1	0	0	0	0
L92;n=0	0	2	0	0	0	0	0	0
L92;n=1	0	2	0	0	0	0	1	0
L92;n=2	0	3	0	0	0	1	0	0
L94	0	3	A8	1	0	0	0	0
L95;n=0	0	3	0	0	0	0	0	0
L95;n=1	0	3	0	0	0	0	1	0
L95;n=2	0	4	0	0	0	1	0	0
L111	0	4	B0	1	0	0	0	0
L112;n=0	0	4	0	0	0	0	0	0
L112;n=1	0	4	0	0	0	0	1	0
L112;n=2	0	5	0	0	0	1	0	0
L115	0	5	B0	1	0	0	0	0
L116;n=0	0	5	0	0	0	0	0	0
L116;n=1	0	5	0	0	0	0	1	0
L116;n=2	0	6	0	0	0	1	0	0
L134	0	6	6C	1	0	0	0	0
L135;n=0	0	6	0	0	0	0	0	0
L135;n=1	0	6	0	0	0	0	1	0
L135;n=2	0	7	0	0	0	1	0	0

Test result

File View

passed

	clk	Program_...	InstrMem...	InstrMem...	IR_output	IR_ctrl	Write_Back	ALU_output
L142;n=0	0	7	0	0	0	0	0	0
L142;n=1	0	7	0	0	0	0	0	0
L152;n=0	0	7	0	0	0	0	1	0
L152;n=1	0	0	0	0	0	1	0	0
L152;n=2	0	0	0	0	A8	0	0	1
L152;n=3	0	0	0	0	A8	0	0	1
L152;n=4	0	0	0	0	A8	0	1	1
L152;n=5	0	1	0	0	A8	1	0	2
L152;n=6	0	1	0	0	A8	0	0	2
L152;n=7	0	1	0	0	A8	0	0	2
L152;n=8	0	1	0	0	A8	0	1	2
L152;n=9	0	2	0	0	A8	1	0	3
L152;n=10	0	2	0	0	A8	0	0	3
L152;n=11	0	2	0	0	A8	0	0	3
L152;n=12	0	2	0	0	A8	0	1	3
L152;n=13	0	3	0	0	A8	1	0	4
L152;n=14	0	3	0	0	A8	0	0	4
L152;n=15	0	3	0	0	A8	0	0	4
L152;n=16	0	3	0	0	A8	0	1	4
L152;n=17	0	4	0	0	A8	1	0	5
L152;n=18	0	4	0	0	B0	0	0	1
L152;n=19	0	4	0	0	B0	0	0	1
L152;n=20	0	4	0	0	B0	0	1	1
L152;n=21	0	5	0	0	B0	1	0	2
L152;n=22	0	5	0	0	B0	0	0	2
L152;n=23	0	5	0	0	B0	0	0	2
L152;n=24	0	5	0	0	B0	0	1	2
L152;n=25	0	6	0	0	B0	1	0	3

Test result

File View

passed

	clk	Program_...	InstrMem...	InstrMem...	IR_output	IR_ctrl	Write_Back	ALU_output
L152;n=12	0	2	0	0	A8	0	1	3
L152;n=13	0	3	0	0	A8	1	0	4
L152;n=14	0	3	0	0	A8	0	0	4
L152;n=15	0	3	0	0	A8	0	0	4
L152;n=16	0	3	0	0	A8	0	1	4
L152;n=17	0	4	0	0	A8	1	0	5
L152;n=18	0	4	0	0	B0	0	0	1
L152;n=19	0	4	0	0	B0	0	0	1
L152;n=20	0	4	0	0	B0	0	1	1
L152;n=21	0	5	0	0	B0	1	0	2
L152;n=22	0	5	0	0	B0	0	0	2
L152;n=23	0	5	0	0	B0	0	0	2
L152;n=24	0	5	0	0	B0	0	1	2
L152;n=25	0	6	0	0	B0	1	0	3
L152;n=26	0	6	0	0	6C	0	0	2
L152;n=27	0	6	0	0	6C	0	0	2
L152;n=28	0	6	0	0	6C	0	1	2
L152;n=29	0	7	0	0	6C	1	0	0
L152;n=30	0	7	0	0	0	0	0	0
L152;n=31	0	7	0	0	0	0	0	0
L152;n=32	0	7	0	0	0	0	1	0
L152;n=33	0	0	0	0	0	1	0	0
L152;n=34	0	0	0	0	A8	0	0	3
L152;n=35	0	0	0	0	A8	0	0	3
L152;n=36	0	0	0	0	A8	0	1	3
L152;n=37	0	1	0	0	A8	1	0	4
L152;n=38	0	1	0	0	A8	0	0	4
L152;n=39	0	1	0	0	A8	0	0	4